

컴퓨터비전 개론 4주차 요약본(6. 1.~6. 8.)

13. 사전 학습 모델_1

13.1 사전학습 모델(Pretrained Model)

- **정의:** ImageNet 같은 대규모 데이터셋으로 미리 훈련된 신경망 모델
- 이미 수백만 장의 이미지를 학습 → 기본적인 시각적 특징(선, 면, 질감)을 인식 가능
- 새로운 문제에 빠르고 정확하게 적응 (학습 시간 단축, 성능 향상)
- 파이토치에서는 `torchvision.models` 로 쉽게 불러올 수 있음

13.2 파인튜닝(Fine-tuning)과 전이학습(Transfer Learning)

- **파인 튜닝 (Fine-Tuning)**
 - 모든 파라미터를 새로운 데이터로 다시 학습
 - 데이터가 **충분히 많을 때** 적합 (예: CIFAR-10의 5만 건)
 - 모델 전체를 새로운 작업에 맞게 조정
- **전이학습(Transfer Learning)**
 - 일부 파라미터는 **고정(동결)**하고 출력 레이어만 학습
 - 데이터가 **적을 때** 더 적합
 - 기존 지식을 최대한 보존
 - 학습 시간이 상대적으로 짧음

13.3 적응형 평균 풀링 (AdaptiveAvgPool2d)

- 입력 이미지 크기와 무관하게 **출력 크기를 일정하게 유지**
- CNN 모델이 다양한 입력 크기에서도 작동할 수 있게 함
- **일반 MaxPool2d:** 커널 크기 지정 → 출력 크기가 입력에 따라 변함
- **AdaptiveAvgPool2d:** 출력 크기 지정 → 입력에 관계없이 일정한 출력

13.4 사전학습 모델 성능 비교

- Top-1은 1순위 예측 오답률 / Top-5는 상위 5개 내 정답 미포함 확률

모델	Top-1 오류(%)	Top-5 오류(%)	특징
AlexNet	43.45	20.91	초창기 CNN
VGG-19-BN	25.76	8.15	안정적, 깊은 구조
ResNet-18	30.24	10.92	가벼움, 잔차 연결
ResNet-152	21.69	5.94	매우 깊은 모델, 높은 성능

13.5 CIFAR-10 데이터셋을 통한 비교

- 이미지 크기 조정

- CIFAR-10 원본: 32×32 픽셀
- 사전 학습 모델 학습 크기: 224×224
- 실습에서 사용: 112×112 (학습 시간과 성능의 균형)

데이터 변환

- 학습 데이터

```
transforms.Compose([
    transforms.Resize((112, 112)), # 리사이즈 (112×112)
    transforms.RandomHorizontalFlip(p=0.5), # 좌우 반전 (p=0.5)
    transforms.ToTensor(), # tensor로 변환
    transforms.Normalize(...), # 정규화
    transforms.RandomErasing(p=0.5) # 랜덤 지우기 (p=0.5)
])
```

- 검증 데이터

```
transforms.Compose([
    transforms.Resize((112, 112)),
    transforms.ToTensor(),
    transforms.Normalize(...)
])
# batch size: 50
```

- **ResNet-18**

```
### 단순 구조, fc 레이어 접근이 쉬움

# 1. 사전 학습 모델 불러오기
net = models.resnet18(pretrained=True)

# 2. 최종 레이어 확인
fc_in_features = net.fc.in_features # 512

# 3. 레이어 교체 (CIFAR-10의 10개 클래스에 맞게)
net.fc = nn.Linear(fc_in_features, n_output)
```

- **VGG-19-BN**

```
### 파라미터가 많고 깊지만, 높은 정확도 달성 가능

# 1. 모델 불러오기
net = models.vgg19_bn(pretrained=True)

# 2. 최종 레이어 확인 (classifier의 6번째 요소)
in_features = net.classifier[6].in_features # 4096

# 3. 레이어 교체
net.classifier[6] = nn.Linear(in_features, n_output)
```

13.6 최적화 함수 비교

- Adam

- 각 파라미터에 대해 학습률을 개별적으로 적응
- 초반 학습 속도가 빠름
- 때로는 파인 튜닝에서 일반화 성능 저하 가능

- SGD with Momentum

- 고정된 학습률 사용
- 이전 기울기의 지수 가중 평균(모멘텀) 활용

- 파인 튜닝에서 더 안정적이고 우수한 일반화 성능
- 부드러운 수렴

항목	Adam	SGD (with Momentum)
방식	적응형 학습률 (1차·2차 모멘트 활용)	일정 학습률 + 관성(모멘텀)
장점	빠른 수렴, 초기 학습 효율적	안정적 수렴, 일반화 성능 우수
단점	Fine-tuning 시 불안정할 수 있음	학습 초반 속도 다소 느림
권장	대규모 데이터 초반 학습	사전학습 가중치 미세조정 시

13.7 ResNet-18 vs VGG-19-BN

- ResNet-18 (현대적 아키텍처)

- 잔차 연결(Residual Connection) 구조
- 파라미터 수가 적어 학습 속도가 빠름
- 가벼운 모델
- 빠른 학습과 우수한 성능

- VGG-19-BN (고전적 아키텍처)

- 합성곱과 풀링을 순차적으로 깊게 쌓음
- Batch Normalization으로 안정성 향상
- 파라미터가 많아 학습 시간이 오래 걸림 (에폭당 약 8분)
- 더 높은 정확도 달성 가능

13.8 핵심 내용 정리

- Fine-tuning & Transfer Learning 요약

- 사전 학습 모델을 활용하면 학습 시간 단축 + 성능 향상
- **Fine-tuning**: 모든 파라미터 재학습 → “모델 전체 최적화”
- **Transfer Learning**: 일부 동결 → “지식 보존 + 효율적 학습”
- 파인 튜닝은 데이터가 충분할 때, 전이 학습은 데이터가 적을 때
- **SGD with Momentum**이 파인 튜닝에 효과적
- **ResNet-18**: 빠른 학습, **VGG-19-BN**: 높은 정확도

- 상황에 맞는 모델 선택이 중요

- 재현성 확보 (`torch_seed`)

```
# 난수 생성 과정을 고정(동일한 시드 설정)하여 동일한 결과 보장
import torch
torch.manual_seed(42)
torch.backends.cudnn.deterministic = True
```

- 풀링 레이어 제거

- CIFAR-10 등 작은 입력 데이터에 맞게 **모델 구조 경량화**
- 리소스 제약 환경에서 특정 환경이나 요구사항에 맞게 모델 구조 최적화 → 연산 최적화
- 새로운 작업의 입력 크기나 특징 추출 방식에 맞춰 조정
- 모델 경량화나 불필요한 연산 제거에 활용

14. 사전 학습 모델 활용_2

14.1 층별 동결 전략 비교

- Full Freeze

- **방법:** Backbone 전체를 동결하고 **Head(마지막 FC 레이어)**만 학습
- **적용**
 - 데이터가 매우 적을 때 (< 1,000장)
 - Pre-training 데이터와 유사한 도메인
- **장점:** 과적합 방지, 빠른 학습
- **단점:** 제한적인 적응력

- Partial Fine-tuning

- **방법:** 후반부 일부 층만 학습, 초반부는 동결
- **적용**
 - 중간 크기 데이터 (1K~10K)

- 도메인 유사도에 따라 학습 범위 조절(도메인 유사도가 높으면 효과↑)
 - 도메인 유사하면 마지막 1~2블록만 Fine-tune
- **장점:** 균형잡힌 성능과 효율성
- **단점:** 최적 동결 지점 찾기 어려움

- Full Fine-tuning

- **방법:** 모든 레이어 학습
- **적용**
 - 큰 데이터셋 (> 10K)
 - Pre-training과 다른 도메인
- **장점:** 데이터가 큰 경우이므로 최대 성능 달성 가능
- **단점:** 과적합 위험, 긴 학습 시간

14.2 차등 학습률 (Discriminative Learning Rates)

- 목적

- 백본(Pre-trained) 층은 **학습률 낮게**
- 새로 만든 Head(FC층)는 **학습률 높게**
 - 사전학습된 특징은 보존
 - 새로운 데이터에 빠르게 적응 가능

PyTorch 구현 포인트

- **Parameter Group** 기능 사용하여 layer별 learning rate 설정

```
optimizer = optim.SGD([
    {'params': model.layer1.parameters(), 'lr': 0.001},
    {'params': model.layer2.parameters(), 'lr': 0.001},
    {'params': model.layer3.parameters(), 'lr': 0.005},
    {'params': model.layer4.parameters(), 'lr': 0.01},
    {'params': model.fc.parameters(), 'lr': 0.01}])
```

14.3 Gradual Unfreezing

Epoch 범위	학습되는 Layer
1~5	Head만 학습
6~10	Head + Layer4
11~15	Head + Layer4 + Layer3
16~	모든 Layer 점진적 해제

- ULMFiT에서 검증된 기법
- 초기에는 안정적으로 Head 학습
- 점진적으로 깊은 층까지 확장
- 급격한 변화 방지 → 안정적인 수렴

14.4 Small Data 최적 학습 프로토콜

Small Data의 정의

- 클래스당 < 100 samples
- 전체 < 1,000 samples
- Pre-training 데이터와 도메인 차이가 큼

Small Data 문제점

- 과적합 위험
- 일반화 성능 저하
- 클래스 불균형 가능성
- 검증 전략의 어려움

2단계 전략

1. 전체 동결 + Head 학습 (Stage 1)
2. 점진적 Layer 해제 + 낮은 LR로 미세조정 (Stage 2)

작은 데이터셋을 위한 강력한 정규화 기법

```
# Weight Decay
optimizer = optim.SGD(model.parameters(),
                        lr=0.001,
                        weight_decay=1e-4) # L2 regularizatio
```

```

n

# Dropout
nn.Dropout(p=0.5) # FC 레이어에 추가

# Early Stopping
patience = 10
if val_loss > best_val_loss: # Validation loss가 n epoch 동안 개선되지 않으면 중단
    patience_counter += 1

# Mixup / CutMix
# 이미지와 레이블을 섞어서 학습
lambda_param = np.random.beta(alpha, alpha)
mixed_image = lambda_param * image1 + (1 - lambda_param) * image2

# Label Smoothing
criterion = nn.CrossEntropyLoss(label_smoothing=0.1) # Hard label(0 or 1) 아닌 Soft label 사용

```

14.5 핵심 내용 정리

- 데이터 크기에 따라 전략을 결정
- 차등 학습률(Differential LR)이 성능 향상에 매우 효과적
- Gradual Unfreezing은 소규모 데이터에 강력한 고급 기법
- Small Data → 먼저 동결 후 점진적 해제 전략 권장
- 전이학습(Pre-trained Model)은 도메인 유사성이 높을수록 효과 극대화
- 상황별 전략 선택 가이드
 - 데이터 적고 도메인 다름 → Full Freeze 중심 + 증강 강화
 - 데이터 중간, 도메인 유사 → Partial Fine-Tuning + Differential LR
 - 데이터 크고 도메인 유사 → Full Fine-Tuning
- 도메인이 다를수록

- 더 많은 층 학습
- 더 낮은 학습률
- 점진적 해제 고려

• 실험과 검증

- Overfitting gap 모니터링
- Validation 성능 중심으로 판단
- Early Stopping으로 과적합 방지

15. 사용자 정의 데이터 활용

15.1 ImageFolder

- 특정 트리 구조로 배치된 이미지 파일을 머신러닝의 입력 데이터로 자동 변환하는 PyTorch 기능
- 특징
 - 폴더 이름이 **자동으로 라벨(label)** 로 매핑
 - 폴더 구조만 맞추면 **바로 Dataset으로 사용 가능**
 - 데이터셋 지정 부분만 변경하면 나머지 학습 코드는 그대로 사용 가능
- 구조

```

root/
├── train/                                # 훈련 데이터 폴더
│   ├── ants/                            # 클래스 1 (폴더명 = 라벨)
│   │   ├── 0013035.jpg
│   │   ├── 1030023514_aad5c608f9.jpg
│   │   └── ...
│   └── bees/                            # 클래스 2
│       ├── 1092977343_cb42b38d62.jpg
│       └── ...
└── val/                                # 검증 데이터 폴더
    ├── ants/
    │   ├── 10308379_1b6c72e180.jpg
    │   └── ...
    └── bees/
        ├── 1032546534_06907fe3b3.jpg

```

```

    ...

### 구조의 규칙 ###
# train/ 폴더와 val/ (검증) 폴더 생성
# 각 폴더 아래에 클래스 이름으로 하위 폴더 생성
# 클래스 폴더 안에 해당 클래스의 이미지 파일 저장

```

15.2 데이터 전처리(Transform) 설정

- **Train Transform (증강, 일반화 향상)**

- `RandomResizedCrop(224)`
- `RandomHorizontalFlip`
- `RandomErasing`
- `ToTensor()`
- `Normalize(mean, std)`

→ 데이터 증강 기법을 사용하여 모델이 매 epoch마다 매번 조금씩 다른 이미지를 학습하게 함

→ **과적합 방지 및 일반화 성능 향상**

- **Validation Transform (고정, 일관된 평가용)**

- `Resize(256)`
- `CenterCrop(224)`
- `ToTensor()`
- `Normalize(mean, std)`

→ 사전학습 된 모델들이 대부분 **224×224** 이미지 기준

→ “항상 같은 부분”으로 평가해야 검증 결과가 일관됨(이미지의 **중앙부**를 사용)

15.3 Dataset & DataLoader 생성

- Train: **244장**
- Val: **153장**
- Classes: **2개 (ants / bees)**

- batch size = 10

```
# 작은 데이터에서는 batch size도 작게 설정함
trainloader = DataLoader(trainset, batch_size=10, shuffle=True)
valloader = DataLoader(valset, batch_size=10)

# shuffle=True: 데이터 순서를 외우는 것 방지 (과적합↓)
```

15.4 Fine Tunning 방식

- **정의:** 사전 학습된 모델의 **모든 파라미터**를 우리의 데이터에 맞게 미세 조정하는 방식
- **구현 단계 (VGG-19-BN 사용)**

```
# Step 1: 사전 학습 모델 불러오기
model = models.vgg19_bn(pretrained=True)

# Step 2: 최종 계층 수정, 분류 클래스가 2개이므로 최종 노드의 출력을 2로 변경
num_features = model.classifier[6].in_features
model.classifier[6] = nn.Linear(num_features, 2) # 2개 클래스

# Step 3: 최적화 함수 설정, 모델의 모든 파라미터를 업데이트 대상으로 지정
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

- **결과**
 - 검증 정확도: 96.08%, 훈련 정확도: 94.26%
 - 과적합 없이 안정적인 성능

15.5 Transfer Learning 방식

- **정의:** 사전 학습 모델의 **대부분의 가중치는 그대로 두고(동결)**, 마지막 분류 계층만 새로 학습하는 방식
- **구현 단계**

```
# Step 1: 가중치 동결, 모든 파라미터 동결
for param in model.parameters():
    param.requires_grad = False

# Step 2: 최종 계층 교체
num_features = model.classifier[6].in_features
model.classifier[6] = nn.Linear(num_features, 2)

# Step 3: 선택적 학습, 최종 계층의 파라미터만 학습
optimizer = optim.SGD(model.classifier[6].parameters(),
                        lr=0.001, momentum=0.9)
```

- 결과
 - 검증 정확도: 약 96%
 - 파인 튜닝과 동일한 성능 기록

항목	파인튜닝	전이학습
업데이트 범위	전체 레이어	마지막 레이어만
데이터 필요량	많을수록 유리	적어도 가능
학습 속도	느림	빠름
과적합 위험	비교적 있음	낮음
권장 상황	데이터 충분	데이터 적음

16. CNN 모델 빌드

16.1 CNN 등장 배경

- 기존 MLP(다층 퍼셉트론)의 한계를 해결하기 위해 CNN이 등장
- CNN은 “이미지를 위한 신경망”으로 설계됨

[MLP의 한계]

- **공간 정보 손실:** 이미지를 1D 벡터로 변환하는 과정에서 픽셀 간의 공간적 관계가 소실
 - 이미지처럼 **공간적 구조(H×W)**를 가진 데이터를 잘 활용하지 못함
- 모든 픽셀이 전부 연결되므로 **파라미터 과다** → 학습 비용 증가 / 과적합 위험 증가

- **위치 불변성 부족:** 동일 객체가 이미지 내 다른 위치 있을 때, 새로운 특징으로 인식 → 평행이동 등에 취약

[CNN의 해결 방식]

1. 지역 연결(Local Connectivity)

- 작은 영역(예: 3×3 , 5×5)만 보고 특징 추출

2. 파라미터 공유(Parameter Sharing)

- 같은 필터(커널)를 전체 이미지에 적용 → 파라미터 획기적으로 감소

3. 계층적 특징 학습(Spatial Hierarchy)

- 초기 층: 에지, 선, 색감
- 깊은 층: 물체 윤곽, 형태

16.2 합성곱 연산(Convolution) 핵심 개념

- 입력 이미지와 필터(커널)의 요소별 곱셈 후 합산 연산으로, 슬라이딩 윈도우 방식을 사용
- 필터와 이미지 영역의 내적(dot product) 결과가 새로운 픽셀 값 생성
- 연산 결과가 모여 **Feature Map(특징맵)** 생성
- 필터 하나는 하나의 "특징"을 잡아내는 역할 (예: 에지 필터, 블러 필터, 색 대비 등)
 - 에지, 코너, 텍스처 등 저수준 특징을 검출하며, 필터 가중치는 역전파 학습을 통해 최적화
- 가장 큰 장점은 **translation invariance**입니다.

16.3 CNN의 구성 요소

- **Filter, Stride, Padding**

요소	설명
Filter(=Kernel)	특징을 추출하는 작은 행렬 (예: 3×3 , 5×5)
Stride	필터가 이동하는 간격 (1 → 촘촘하게, 2 → 더 크게 이동)
Padding	이미지 경계를 보존하기 위해 가장자리에 0을 채움

- **필터/커널 (Filter/Kernel):** 주로 3×3 크기를 사용하며, 개수는 출력 Feature Map의 채널 수를 결정

- **스트라이드 (Stride):** 필터 이동 간격 제어, 큰 스트라이드는 계산량 줄이지만 정보 손실이 발생할 수 있음
- **패딩 (Padding):** 입력 주변에 0을 추가하여 출력 크기를 제어, Same Padding은 출력 크기를 입력과 동일하게 유지

→ Padding='same' → 입력과 출력의 크기를 동일하게 유지

→ Stride↑ → 출력 이미지 크기↓ (계산량 감소)

- **Feature Map (특징맵)**

- 커널이 감지한 패턴(선·모서리·색 변화 등)을 담아낸 출력
- CNN이 학습함에 따라 필터가 점점 더 복잡한 패턴을 학습
- **중간층(Conv Layer)에서 무엇을 정확히 보고 있는지 나타내는 결과**

- **활성화 함수 (Activation Function)**

- CNN에서 가장 많이 쓰는 함수: **ReLU**
- 음수 → 0
- 양수 → 그대로
- 계산이 빠르고 Gradient Vanishing 문제를 완화

비선형성(Non-Linearity) 부여 → "딥러닝"의 핵심 조건

- **Pooling**

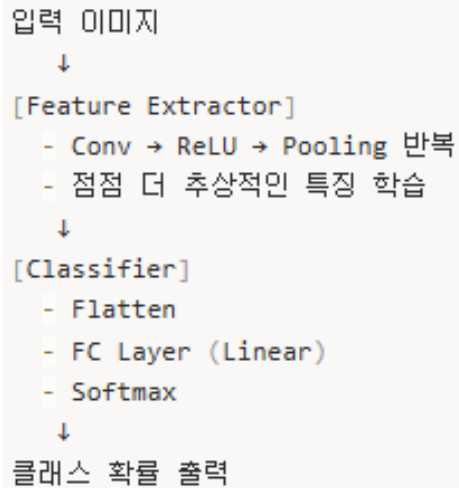
- **정의:** 합성곱으로 얻은 Feature Map을 **축소**하는 과정
- **역할:** 차원 축소 (계산량 감소), 위치 불변성 학습, 수용 영역 확대

풀링 종류	역할
Max Pooling	가장 큰 값을 선택 → 가장 강한 특징만 남김
Average Pooling	평균 값으로 축소, 윈도우 내 평균값을 계산하며, Global Average Pooling은 분류기 직전에 사용
Global Average Pooling (GAP)	특징맵 전체를 하나의 스칼라로 축소

- **목적**

- 공간 크기 감소
- 연산량 감소
- 위치 변화에 강인한 모델(translation invariance) 형성

16.4 CNN 전체 구조 흐름



PyTorch로 CNN 모델 기본 구성 구현 예시

특징 추출부

```
self.features = nn.Sequential(  
    nn.Conv2d(3, 32, 3, padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(2),  
    nn.Conv2d(32, 64, 3, padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(2)  
)
```

분류기

```
self.classifier = nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(64*8*8, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10)  
)
```

• CNN 하이퍼파라미터 설계 포인트

- 깊이가 깊어질수록 채널 수는 증가시키고(예: 32 → 64 → 128 → 256), 풀링을 통해 공간 크기는 점진적으로 축소 → 필터 수 증가 규칙: 32 → 64 → 128 → 256 (깊

어질수록 특징 수 증가)

- 필터 크기: 대부분 3×3 가장 일반적
- Stride / Padding: 보통 stride=1, padding=1
- Pooling은 2×2 stride 2
- Dropout 여부: 과적합 방지에 필수 (특히 FC층)

16.5 CNN 학습 과정

1. 배치 단위로 데이터 불러오기
2. 모델에 넣어 예측(pred) 생성
3. 손실(loss) 계산 (CrossEntropyLoss)
4. 역전파(backpropagation)
5. optimizer.step()로 파라미터 업데이트
6. 반복(epoch)
 - **손실 함수:** CrossEntropyLoss (다중 클래스 분류)
 - **옵티마이저:** Adam (Adaptive learning rate)
 - **정규화 기법:** Dropout (FC layer, p=0.5), Batch Normalization (Conv layer 후), Data Augmentation

[Train Mode / Eval Mode 구분]

- model.train() → Dropout 활성화
- model.eval() → BatchNorm/Dropout 비활성 + no_grad()

16.6 핵심 포인트 정리

- **CNN 핵심 구조**
 - Conv → ReLU → Pool → FC
 - **Feature Extractor (특징 추출기):** Conv + ReLU + Pool 블록으로 구성, 이미지 유용한 특징을 추출
 - **Classifier (분류기):** Fully Connected Layer(FC)로 구성되며, 추출된 특징으로 클래스를 예측
- **CNN의 3대 핵심 개념**
 - 지역 연결

- 파라미터 공유
- 계층적 특징학습
- **기본 모듈**
 - Conv2d(필터)
 - ReLU(비선형)
 - MaxPool(크기 축소)
 - Flatten + Linear(Final Classifier)
- **아키텍처 설계 규칙**
 - 필터 수는 점진적으로 증가
 - 3×3 Conv + 2×2 Pool이 기본
 - Feature Extractor + Classifier 구조 분리
- **PyTorch 구현 패턴**
 - `nn.Sequential` 로 특징추출부 구성
 - `forward()` 에서 features → classifier 순차 실행
- **모델 평가 및 시각화:**
 - **학습 곡선:** Loss와 Accuracy 변화로 과적합 여부를 판단
 - **Confusion Matrix:** 클래스별 혼동을 확인
 - **Feature Map:** 네트워크가 학습한 에지나 고수준 패턴을 이해

17. Natural Language Processing (NLP)

17.1 자연어처리란 무엇인가?

- NLP는 인간의 언어(텍스트)를 컴퓨터가 이해·생성하도록 하는 기술
- 딥러닝을 활용해 문장을 분석하고 의미를 파악하며 다양한 언어적 작업을 가능하게 한다
- **주요 작업 영역**
 - **토큰화(Tokenization):** 문장을 단어/서브단어로 분리
 - **품사 태깅(POS Tagging):** 단어의 문법적 역할 파악

- **개체명 인식(NER):** 사람·조직·장소 등 엔티티 인식
- **감성 분석(Sentiment Analysis):** 문장 감정(긍정/부정) 분류
- **실제 응용 분야:** 챗봇, 검색, 리뷰 분석, 고객 피드백 분석 등

17.2 텍스트 전처리 파이프라인 & 단어 임베딩의 이해

• 전처리 단계

1. **토큰화** – 단어/서브워드 분리
2. **정제(Cleaning)** – 특수문자 제거, 소문자화 등
3. **정수 인코딩** – 단어에 인덱스 부여
4. **패딩(Padding)** – 길이 맞추기
5. **임베딩 변환** – 단어 → 벡터화

• 실제 예시

- 문장: "I loved the movie!"
 - 토큰: [I, loved, the, movie, !]
 - 정수 인덱스: [12, 45, 8, 99, 3]
 - 패딩 결과: [12, 45, 8, 99, 3, 0, 0, 0]

• 단어 임베딩 이해

- **원핫 인코딩(One_Hot):** 차원↑ 비효율적, 단어 의미 관계 X
- **밀집 임베딩(Dense Embedding)**
 - 단어 의미를 벡터 공간에 위치시킴
 - 비슷한 의미의 단어는 가까운 위치
 - PyTorch: `nn.Embedding(vocab_size, embedding_dim)`
 - 임베딩 차원 보통 **128, 256, 300** 등
- **임베딩의 주요 특성**
 - 의미적 유사성 반영
 - 단어 간 거리 계산 가능
 - 다양한 다운스트림 작업에서 재사용 가능

17.3 LSTM (Long Short-Term Memory)

LSTM은 기존 RNN의 기울기 소실 문제를 해결하여 중요한 정보를 장기간 유지할 수 있도록 설계됨

- LSTM의 주요 개념
 - 셀 상태(Cell State)
 - RNN의 hidden state와 비슷하지만, 장기적인 정보 유지 가능
 - 게이트 (Gates)
 - LSTM의 주요 요소로 정보를 추가하거나 제거하는 역할 수행
 - 입력 게이트 (Input Gate): 새로운 정보를 셀 상태에 추가할지 결정
 - 망각 게이트 (Forget Gate): 이전 정보를 유지할지 삭제할지 결정
 - 출력 게이트 (Output Gate): 최종 출력값 결정, 어떤 정보를 다음으로 보낼지 결정
- LSTM 활성화 함수
 - 시그모이드(σ) 함수 → 0~1 사이 값을 출력해 가중치 조정
 - tanh 함수 → -1~1 범위의 값을 반환하여 RNN의 장기 의존성을 학습
- LSTM의 장점
 - 장기 의존성 처리
 - 문맥 기반 처리 가능
 - 시계열·텍스트 등 순서가 중요한 데이터에 적합

17.4 GRU (Gated Recurrent Unit)

LSTM의 간소화 버전으로, 비슷한 퍼포먼스를 가지고 있는데 연산량이 감소

- GRU의 주요 특징
 - LSTM과 유사한 기능 수행(기울기 소실 문제 해결)
 - 망각 게이트와 입력 게이트를 하나로 합쳐 연산량 감소
 - 적은 파라미터 수로도 높은 성능 유지 가능
 - 비교적 작은 데이터 셋에서도 좋은 성능 발휘

17.5 Transformer로의 전환

- RNN/LSTM의 한계
 - 순차 처리로 병렬 처리 어려움

- 긴 문장을 처리할수록 기억력 저하
- 연산 시간 길어짐
- **Transformer 장점**
 - 병렬 처리 가능 → 효율적
 - 장거리 의존성 처리 우수
 - 더 많은 데이터로 학습 가능
 - 큰 모델에도 적합

17.6 DistilBERT 개요

- **DistilBERT란?**
 - BERT의 경량화 모델
 - **파라미터 40% 감소, 속도 60% 증가**, 성능은 97% 유지
 - 지식증류(Knowledge Distillation) 기법 활용
- **아키텍처 특징**
 - 6개 레이어 (BERT-base는 12개)
 - Hidden size 768
 - vocab 30k+
 - 모델 크기 약 66M
- **DistilBERT 학습 단계**
 - 사전 학습(대규모 코퍼스)
 - 분류 헤드 추가
 - 파인튜닝
 - 평가 및 배포
- **Bi-LSTM 장단점과 DistilBERT 장단점 비교**

[Bi-LSTM]

- **장점**
 - 구조 단순
 - 적은 데이터에서도 학습 가능

- 빠른 학습·추론
- 단점
 - 장거리 문맥 이해 한계
 - 최신 Transformer 대비 성능 뒤처짐

[DistilBERT]

- 장점
 - 강력한 문맥 이해
 - 대규모 데이터 학습 가능
 - 높은 정확도
 - Transfer Learning 가능
- 단점
 - 더 많은 자원 필요
 - LSTM보다 추론 느림

- 참고 사항 -

어텐션 (Attention)

입력의 모든 요소를 동일하게 처리하지 않고, 중요한 부분에 더 많은 가중치를 할당

- 어텐션의 필요성
 - Seq2Seq는 입력 문장을 고정된 벡터로 변환하기 때문에 긴 문장 학습이 어려움
 - Attention을 사용하면, 문장에서 중요한 단어를 더 강조하여 가중치를 다르게 적용
 - 예) "I love you"를 "나는 너를 사랑해"로 번역할 때, 각 단어가 어떻게 매핑되는지 계산 가능

▶ 어텐션 (Attention)

